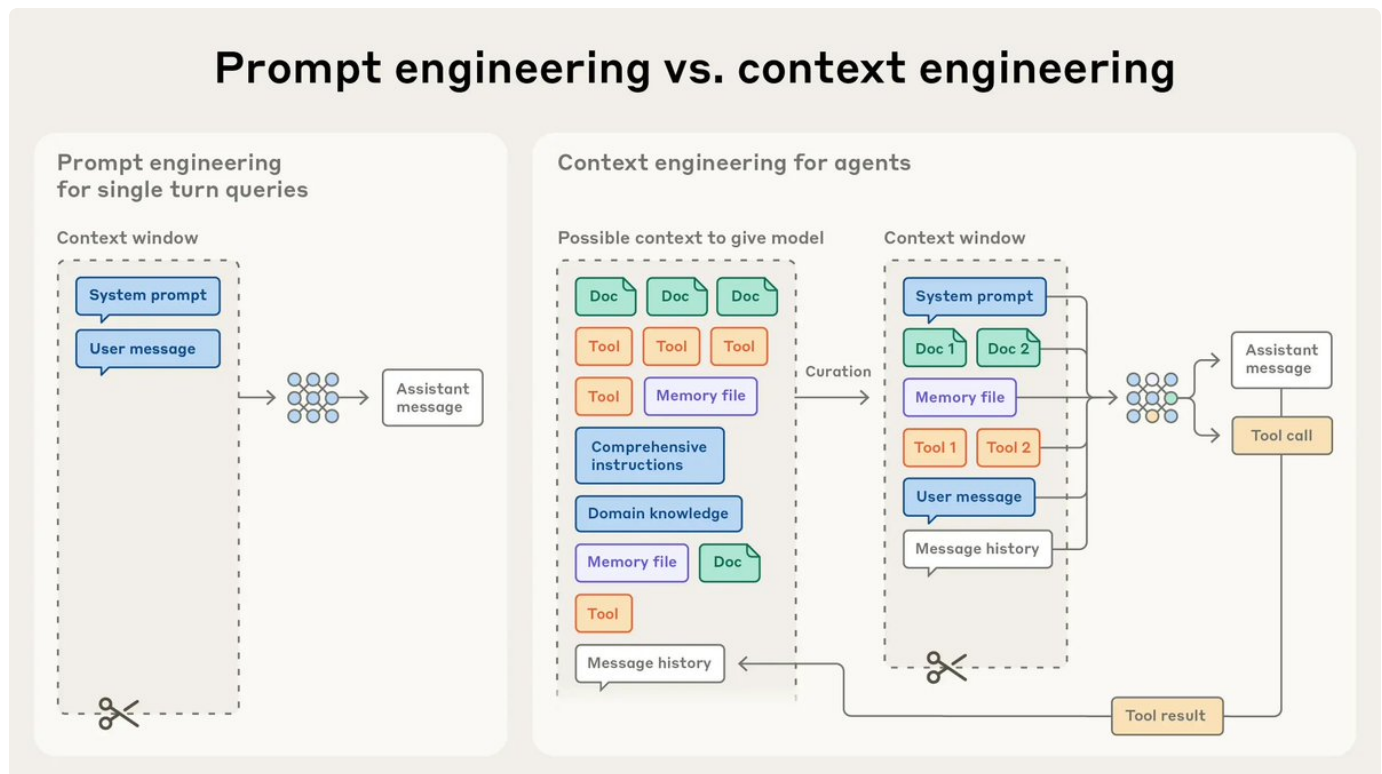


上下文工程决定了稳定（Agent学习笔记）

1. 上下文为什么要分层
2. 压缩策略
3. Prompt Caching
4. Skills
5. 压缩最容易丢掉什么
6. 文件系统为什么适合做上下文接口

from: <https://x.com/HiTw93/status/2034627967926825175>



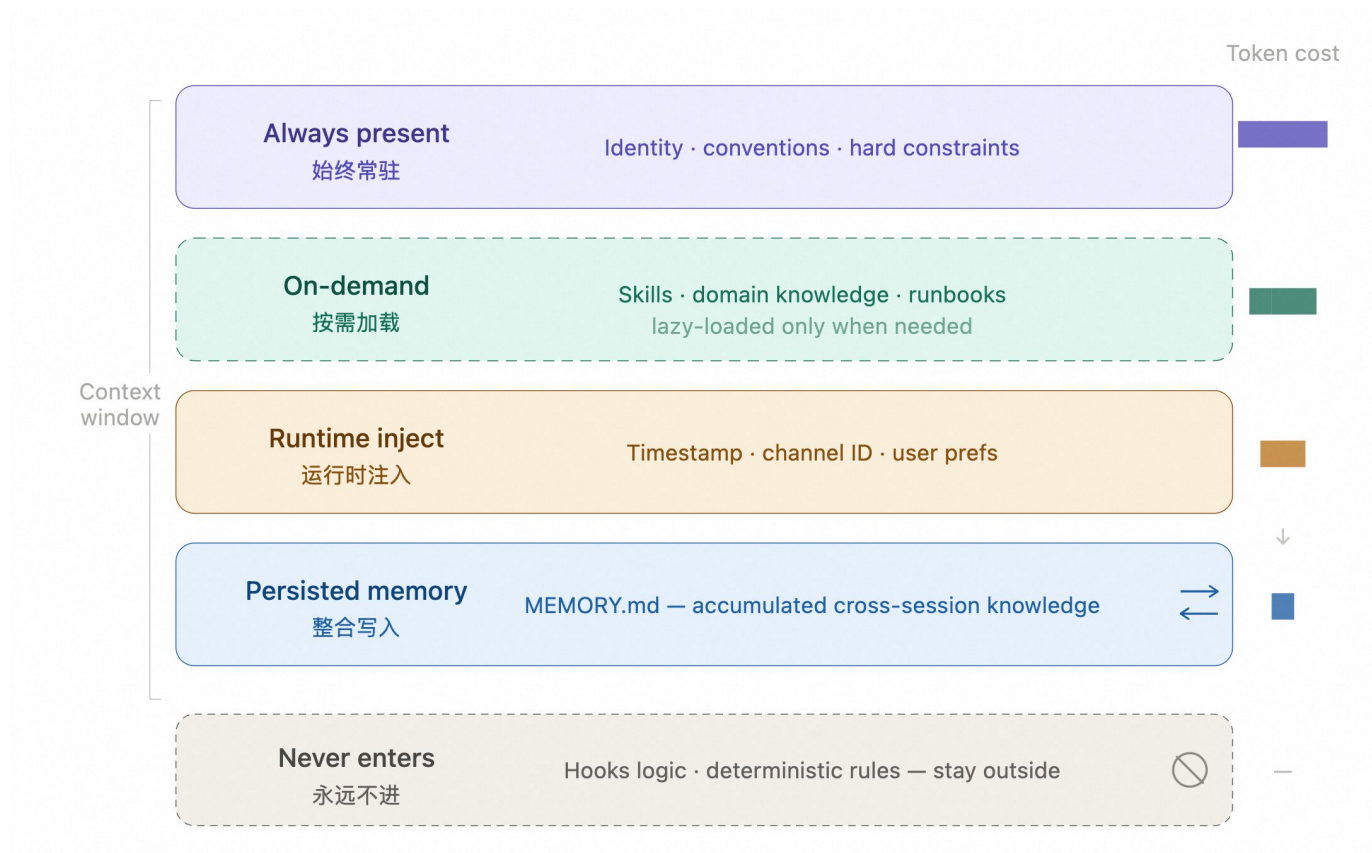
上下文工程的关键，不是窗口够不够长，而是放进去的东西是否真正相关（

此处为语雀内容卡片，点击链接查看：

https://www.yuque.com/crisschan/emow0v/lwp9c0i3gzy63yaf?view=doc_embed

），上下文越长，关键信号越容易被噪声稀释。Context Rot（上下文腐化），因为上下文中无关信息太多了，导致Agent的决策质量明显下滑。很多看起来像模型能力不足的问题，往往可以追溯到上下文组织不当。

1. 上下文为什么要分层



- 常驻层：身份定义、项目约定、绝对禁止项等稳定规则
- 按需加载：Skills，领域知识和操作流程
- 运行时注入：当前时间、渠道 ID、用户偏好等动态信息
- 记忆层：持久化记忆，跨会话经验写入 MEMORY.md
- 系统层：Hooks 或代码规则处理确定性逻辑

别把确定性逻辑放进上下文，凡是可以通过 Hooks、代码规则或工具约束表达的内容，都应交给外部系统处理，而不是让模型反复读取。（Openclaw也是类似的设计）

2. 压缩策略

压缩的目标不是单纯减少 token，而是在有限上下文预算内优先保留决策价值最高的信息，并把可重建内容移出上

- 滑动窗口：丢弃旧消息，成本极低，会丢早期上下文，适合简短对话。实现最简单，超过阈值直接丢弃旧消息，适合短对话或低风险任务，但会同时丢掉早期决策背景。
- LLM 摘要：模型生成总结，成本中等，丢细节保留决策，更适合长任务，常见做法是在上下

文接近容量时触发整合，把旧消息摘要写入 MEMORY.md，保留原始记录用于追溯。进阶做法是 branch summarization，在摘要时明确保留架构决策、未完成任务和关键约束。

- 3. 工具结果替换：占位符替换原始输出，成本极低，适合工具调用密集的 Agent，工具输出不再保留原始内容，而是用占位符或摘要替换，例如 micro_compact（每轮替换旧工具输出）、auto_compact（上下文超阈值时自动触发归档并摘要），这种方式通常开销最低，因为它主要处理占比最大的工具输出，而不影响核心决策信息。

3. Prompt Caching

很多 Agent 的系统提示都很长，但其中大部分内容在整个会话里基本不变，每轮请求都重新编码，等于在重复支付同一段输入成本。

Anthropic API 支持对消息内容块标记 `cache_control: { type: "ephemeral" }`。首次请求会建立缓存，之后 5 分钟内，相同前缀的请求可以直接复用。被缓存部分的费用可下降约 90%，很适合系统提示较长、调用又频繁的 Agent。是否命中缓存，可以通过 `response.usage` 里的 `cache_read_input_tokens` 和 `cache_creation_input_tokens` 来判断。

4. Skills

Skills 是上下文工程里非常有效的一种模式，核心思路是：**系统提示只保留索引，完整知识按需加载。**

Skill 描述要短，因为描述本身会常驻上下文，几十个 token 的差异在高频调用里会持续累积。Skill 描述要写成路由条件，而不是功能介绍。至少要说明三件事：什么时候用、什么时候不要用、产出物是什么。最直接的写法是加入 `Use when / Don't use when`，再补几条反例。很多路由失败不是模型能力问题，是边界写得不清楚。系统提示里也要把调用规则写明确，**每次回复前先扫描 available_skills，有明确匹配时再读取对应 SKILL.md，多个匹配时优先选最具体的那个，没有匹配就不读取，一次只加载一个，重点不是给模型更多自由，而是把路由过程压缩成一个低成本、可重复执行的步骤。**



注意：Skills 不能等 Agent 想起来再用，而要每轮都先扫描描述，不过扫描成本要足够低，实际加载的 Skill 数量也要受控。如果 Skill 会触发外部 API 写操作，系统提示里应显式补充速率限制要求，例如尽量批量写入、避免逐条循环、遇到 429 主动等待。（在 HTTP 协议中，响应状态码 429 Too Many Requests 表示在一定的时间内用户发送了太多的请求，即超出了“频次限制”。）

Skills 和 MCP 在上下文成本上的特征并不相同。很多 MCP 调用会直接把完整结果返回给模型，模型无法在返回前过滤，因此上下文预算会被迅速吃掉。相比之下，CLI + 单句描述的 Skill 更接近模型熟悉的调用方式，在无状态数据获取场景里通常更容易组合，也更容易压缩。当然 MCP 也有明确适用场景，例如 Playwright 这类需要维护状态的任务，但对大多数可过滤、可拼接的数据读取任务，CLI 往往更简洁。

5. 压缩最容易丢掉什么

压缩阶段最常见的问题，不是摘要不够短，而是保留顺序设错了。LLM 通常会优先删除那些看起来还可以重新获取的信息，早期的 tool output 通常最先被移除，但与之相关的架构决策、约束理

由和失败路径也很容易一并丢失。最好在 CLAUDE.md 或等价文档里明确写出压缩时的保留优先级：

```
1  ### Compact Instructions 如何保留关键信息
2  保留优先级：
3  1. 架构决策，不得摘要
4  2. 已修改文件和关键变更
5  3. 验证状态，pass/fail
6  4. 未解决的 TODO 和回滚笔记
7  5. 工具输出，可删，只保留 pass/fail 结论
```

另一个常被忽略但很重要的要求是，压缩时不要改动各种标识符。像 UUID、hash、IP、端口、URL、文件名这类值，都必须原样保留，不能改写、简化，也不能凭感觉修正，这个约束看起来很细，但一旦把 PR 编号或 commit hash 改错一位，后续工具调用就会直接失效。

6. 文件系统为什么适合做上下文接口

只要 Agent 具备按需拉取信息的能力，初始上下文越克制，整体效果往往越稳定。Cursor 把这种方式称为 Dynamic Context Discovery，核心不是预先提供尽可能多的信息，而是默认少给，只在需要时读取。

这也是文件系统会成为优质上下文接口的原因。工具调用经常返回大量 JSON，几次搜索就足以堆出成千上万 token。与其在上下文中截断、粘贴或长期保留，不如直接写入文件，让 Agent 通过 grep、rg 或脚本按需读取。工具写文件，Agent 读文件，开发者也可以直接查看文件，这比让大段原始输出在上下文里反复流转要干净得多。

Cursor 在 MCP 工具上也验证过这个方向：他们把工具描述同步到文件夹，Agent 默认只看到工具名，需要时再查询具体定义，A/B 测试中，调用 MCP 工具的任务总 token 消耗减少了 46.9%。

同样的思路也适用于长任务压缩。压缩触发时，不直接丢弃历史，而是把聊天记录完整保留为文件，摘要里只引用文件路径。后续如果 Agent 发现摘要缺少细节，仍然可以回到历史文件里检索。这样压缩就变成了一种有损但可追溯的操作，而不是一次不可恢复的硬截断